Acme Inventory Service - Architecture Guide

1. Architecture Overview

The Acme Inventory Service follows a three-tier architecture: a REST API layer, a business service layer, and a PostgreSQL persistence layer. Clients communicate over HTTPS using JSON payloads.

An NGINX reverse proxy terminates TLS and forwards requests to the Inventory API on port 8080. A Redis cache stores frequently read product data to reduce database load.

Stock-change events are published asynchronously to a RabbitMQ message queue and consumed by the Notification Worker, which sends low-stock email alerts.

2. Components

API Gateway (NGINX): terminates TLS and applies rate limiting of 100 requests per second per client.

Inventory API: a stateless Java Spring Boot application that exposes the /api/v1 REST endpoints.

Stock Service: implements business rules such as stock reservations and low-stock thresholds.

Notification Worker: consumes stock events from RabbitMQ and emails an alert when the stock of a product falls below the configured threshold.

PostgreSQL 16: the primary data store. Backups run nightly at 02:00 UTC and are retained for 14 days.

Redis 7: the product cache, with a default time-to-live of 300 seconds.

3. Deployment and Scaling

In production the service is deployed as Docker containers orchestrated by Kubernetes. Each component runs in its own Kubernetes deployment. For single-server installations, see the Installation Guide.

The Inventory API runs a minimum of 2 replicas. A Horizontal Pod Autoscaler scales it from 2 to 10 replicas when average CPU usage exceeds 70 percent.

Releases use rolling updates with maxUnavailable set to 0, so deployments cause no downtime.

Configuration is supplied through Kubernetes ConfigMaps, and credentials through Kubernetes Secrets.

A liveness probe calls GET /api/v1/health every 10 seconds; a pod is restarted after 3 failed checks.

Disaster recovery: restore the latest PostgreSQL backup. The recovery point objective is 24 hours and the recovery time objective is 4 hours.